

Pseudocode for IDW and ILDW Baselines

IDW (midpoint-based, truncated)

Algorithm 1 IDW_Field_From_Est_Input

Require: `est_input_json`, $r_{\max}$, power p , ε , default value d_0 , optional `link_values`

```

1: Read header, links from est_input_json
2: Compute midpoint of each link:  $m_k = ((x0_k + x1_k)/2, (y0_k + y1_k)/2)$ 
3: if link_values is not provided then
4:    $A_k \leftarrow$  link attenuation A_db,  $L_k \leftarrow$  link length in km
5:    $(k_k, \alpha_k) \leftarrow \text{itu838\_k\_alpha}(f_k, \text{pol}_k)$ 
6:    $R_k \leftarrow \text{link\_rain\_from\_attenuation}(A_k, L_k, k_k, \alpha_k)$ 
7: else
8:    $R_k \leftarrow$  provided link_values
9: end if
10: Build query points  $q_t$  as all pixel centers from header
11: Build KD-tree on  $\{m_k\}$ 
12: for each pixel center  $q_t$  do
13:   Initialize  $\hat{R}_t \leftarrow d_0$ 
14:    $I_t \leftarrow \{k : \|m_k - q_t\| \leq r_{\max}\}$  via radius query
15:   if  $I_t$  is empty then
16:     continue
17:   end if
18:   Compute distances  $\delta_k = \|m_k - q_t\|$  for  $k \in I_t$ 
19:   if any  $\delta_k \leq 0$  then
20:      $\hat{R}_t \leftarrow R_k$  of first exact hit; continue
21:   end if
22:    $\delta_k \leftarrow \max(\delta_k, \varepsilon)$ 
23:    $w_k \leftarrow \delta_k^{-p}$ 
24:   if  $\sum_{k \in I_t} w_k \leq 0$  or not finite then
25:     continue
26:   end if
27:   Normalize  $w_k \leftarrow w_k / \sum_j w_j$ 
28:    $\hat{R}_t \leftarrow \sum_{k \in I_t} w_k R_k$ 
29: end for
30: Reshape  $\hat{R}$  to  $(H, W)$  and return  $(\hat{R}, R)$ 

```

ILDW (segment-distance + crossing rule)

Algorithm 2 ILDW_Field_From_Est_Input

Require: `est_input_json`, $r_{\max}$, power p , ε , default value d_0 , optional `link_values`

```
1: Read header, links, segments_by_link from est_input_json
2: Compute link endpoints  $(a_k, b_k)$  and midpoints  $m_k$ 
3: if link_values is not provided then
4:   Compute  $R_k$  from  $(A_k, L_k, f_k, \text{pol}_k)$  exactly as in IDW
5: else
6:    $R_k \leftarrow$  provided link_values
7: end if
8: Build list  $C_t$ : crossing links per pixel from segments_by_link
9:  $r_{\text{search}} \leftarrow r_{\max} + \max_k \|b_k - a_k\|/2$ 
10: Build KD-tree on link midpoints  $\{m_k\}$ 
11: Build all pixel centers  $q_t$ ; initialize  $\hat{R}_t \leftarrow d_0$ 
12: for each pixel index  $t$  do
13:   if  $C_t \neq \emptyset$  then
14:     Deduplicate  $C_t$  (if needed), then set  $\hat{R}_t \leftarrow \text{mean}\{R_k : k \in C_t\}$ 
15:     continue
16:   end if
17:   Candidate links  $I_t \leftarrow \{k : \|m_k - q_t\| \leq r_{\text{search}}\}$  via midpoint radius query
18:   if  $I_t$  is empty then
19:     continue
20:   end if
21:   Compute exact point-to-segment distances  $\delta_k = \text{dist}(q_t, [a_k, b_k])$  for  $k \in I_t$ 
22:   Keep only  $J_t = \{k \in I_t : \delta_k \leq r_{\max}\}$ 
23:   if  $J_t$  is empty then
24:     continue
25:   end if
26:   if any  $\delta_k \leq 0$  in  $J_t$  then
27:      $\hat{R}_t \leftarrow R_k$  of first exact hit; continue
28:   end if
29:    $\delta_k \leftarrow \max(\delta_k, \varepsilon)$  for  $k \in J_t$ 
30:    $w_k \leftarrow \delta_k^{-p}$ ; if  $\sum w_k$  invalid then continue
31:   Normalize and set  $\hat{R}_t \leftarrow \sum_{k \in J_t} w_k R_k$ 
32: end for
33: Reshape  $\hat{R}$  to  $(H, W)$  and return  $(\hat{R}, R)$ 
```
